



# Student Performance Prediction AI

A machine learning system that predicts Pass / Fail outcomes  
from attendance, test scores, GPA, and study habits

**Abdur Rahman**

Computer & Communication Engineering, IIUC

# Why Predict Student Performance?

Catching risk early turns a semester-end surprise into a solvable problem.



## Early Intervention

Flag at-risk students weeks before finals, while support still has time to work.



## Targeted Resources

Help faculty and advisors focus tutoring and mentoring where it matters most.



## Evidence Over Guesswork

Replace gut-feeling assessments with a model trained and tested on real patterns.

# The Original Model Had a Fatal Flaw

The dataset's labels were generated from the exact formula the model tried to predict.

## BEFORE

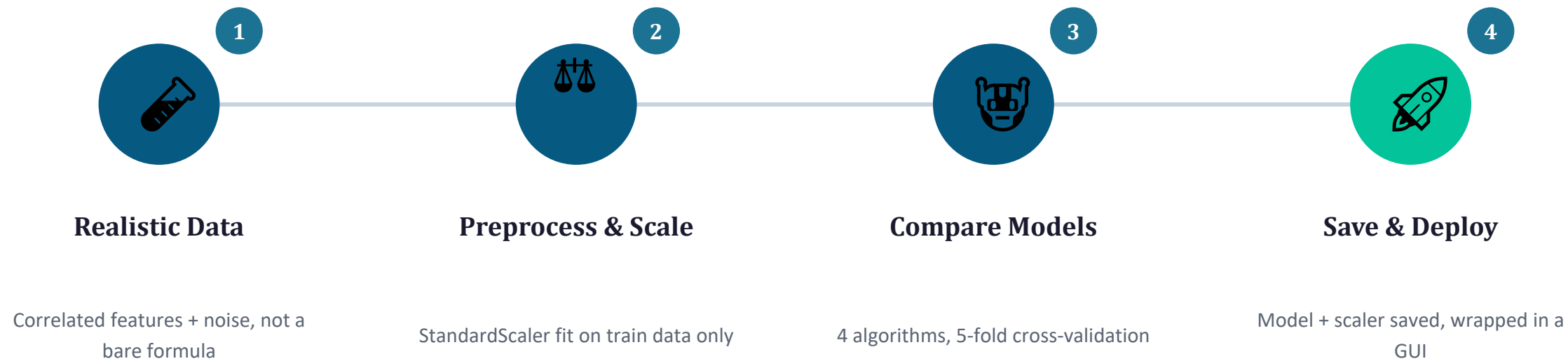
- X Labels computed directly from a fixed formula  
( $2 \times \text{attendance} + 3 \times \text{CT} + \text{mid} + 5 \times \text{GPA}$ )
- X Model just reverse-engineered known arithmetic — not real learning
- X 98% accuracy, but practically meaningless
- X 78% Pass / 22% Fail — heavily imbalanced classes

## AFTER

- ✓ Labels from correlated, noisy features that mimic real student behaviour
- ✓ 4% intentional label noise, like real grading inconsistency
- ✓ 85% accuracy — honestly earned on data the model never saw
- ✓ 57% Pass / 43% Fail — realistically balanced classes

# Rebuilding It the Right Way

Four stages take the project from a toy script to a defensible ML system.



# A Dataset That Behaves Like Real Students

## 5 Input Features



Attendance

0 – 10



CT Marks

0 – 10



Mid Marks

0 – 30



Previous Semester GPA

0 – 4.0



Study Hours / Week

0 – 40

## How It Stays Realistic

- A hidden “ability” factor per student drives every feature, so attendance, marks, and GPA correlate the way real students' do
- Each feature keeps its own independent random noise on top
- 4% of pass/fail labels are randomly flipped to mimic real-world grading inconsistency
- 1,200 students simulated — 57% Pass / 43% Fail

# From Raw Data to Trained Model

Every stage guards against the leakage and overfitting that made the original version misleading.



*Candidate models: Logistic Regression • Random Forest • Gradient Boosting • SVM (RBF kernel)*

The winner is chosen by 5-fold cross-validation F1 score on the training data — not by which model happened to get lucky on the test split.

# Four Models, Compared Honestly

Cross-validation F1 (train-only signal) vs. Test F1 — close agreement means no lucky test split.



# How Good Is It, Really?

Measured on 240 students the model never saw during training

**85.0%**

Accuracy

**86.2%**

Precision

**87.5%**

Recall

**86.9%**

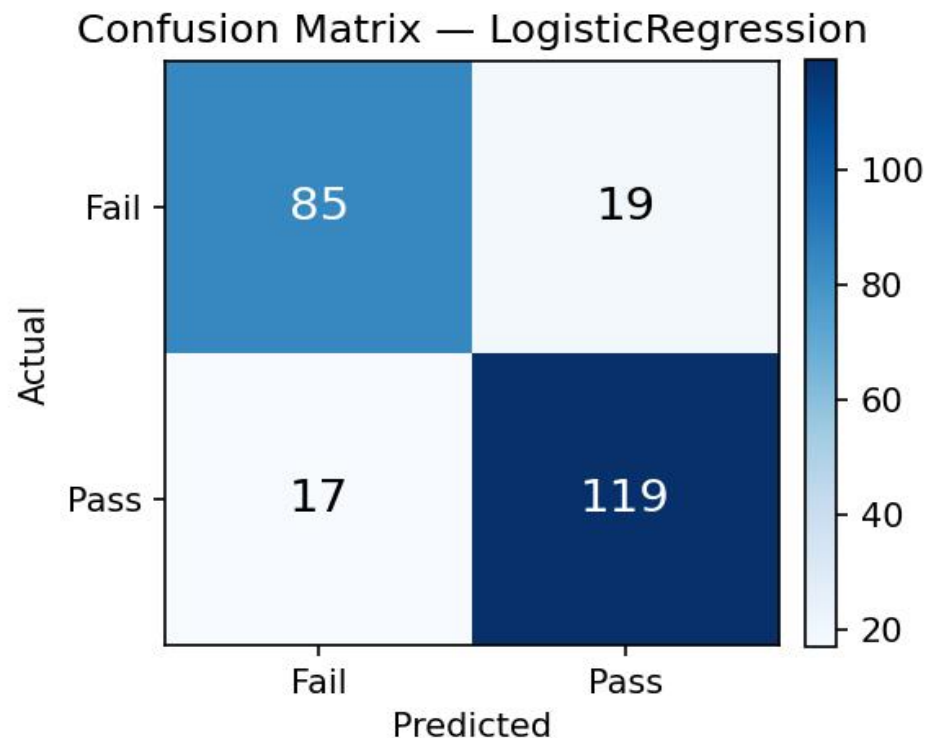
F1 Score

**92.1%**

ROC-AUC

*Precision, Recall, and F1 all land within 2 points of each other — the model isn't skewed toward calling everyone a Pass or a Fail.*

# Where the Model Gets It Right — and Wrong



**119** Correctly predicted Pass

**85** Correctly predicted Fail

**19** False Positives — predicted Pass, actually Fail

**17** False Negatives — predicted Fail, actually Pass

*Recall (87.5%) beats precision (86.2%) — the model is slightly more willing to flag a risk than to miss one, which fits an early-warning use case.*

# A Real Desktop App — Not Just a Script

Built with Tkinter + ttkbootstrap for a modern, card-based interface.

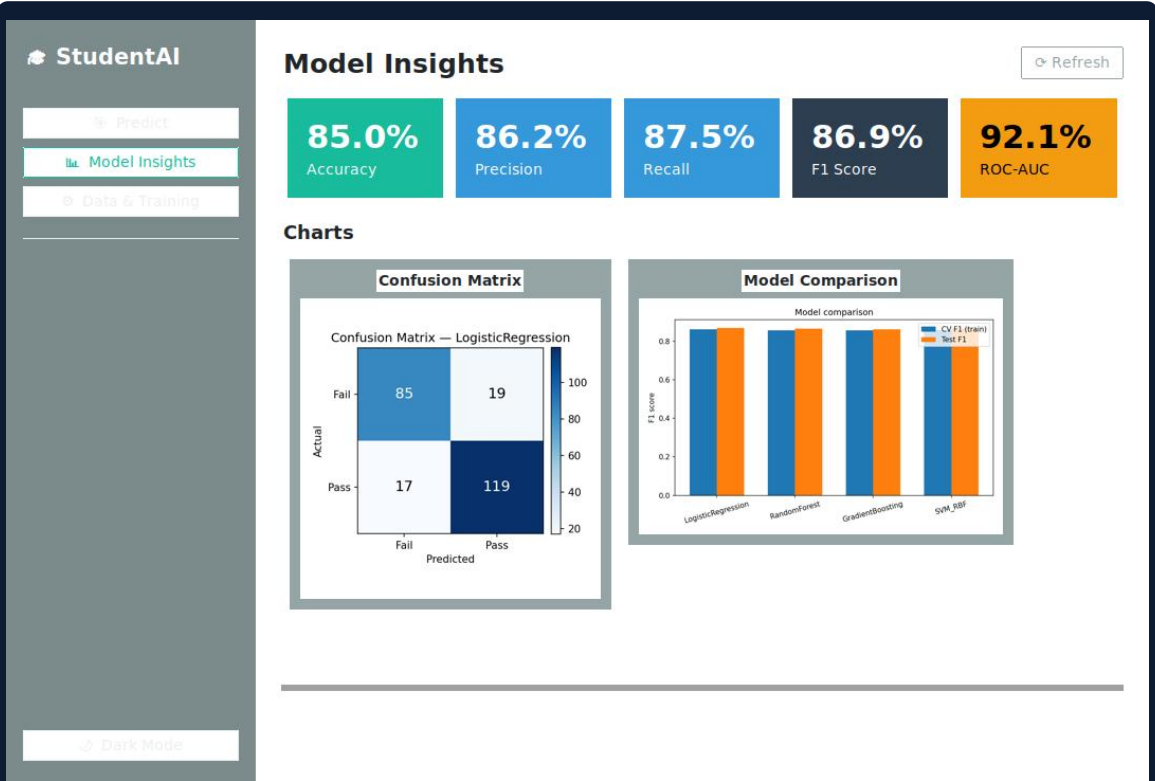
- ▶ Sliders + precise spinboxes for all 5 inputs, always in sync
- ▶ One-click prediction with a color-coded Pass / Fail result card
- ▶ Live confidence bar showing the model's own certainty
- ▶ Shows which model is active and its test accuracy

“Predict” tab

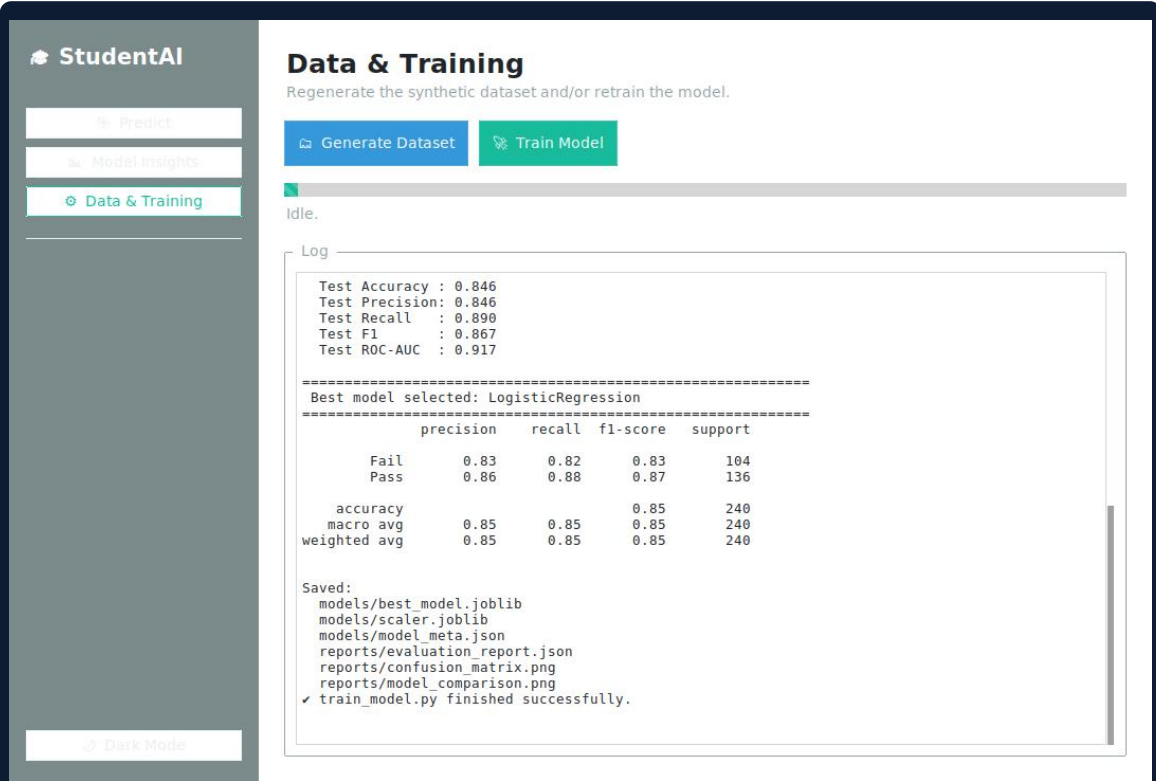
The screenshot displays the 'StudentAI' desktop application. On the left is a dark sidebar with three tabs: 'Predict' (active), 'Model insights', and 'Data & Training'. At the bottom of the sidebar is a 'Dark Mode' toggle. The main content area is titled 'Predict Student Outcome' with a subtitle 'Enter the student's stats to get a Pass/Fail prediction.' Below this, under 'Student Data', are five input controls: 'Attendance' (slider from 0 to 9.0, value 9), 'CT Marks' (slider from 0 to 9.0, value 9), 'Mid Marks' (slider from 0 to 20.0, value 20), 'Previous Semester GPA' (slider from 0 to 3.27, value 3.27), and 'Study Hours / Week' (slider from 0 to 5.0, value 5). A green 'Predict Result' button is at the bottom of this section. To the right, a large green card shows a white checkmark, the word 'PASS', and the text 'This student is predicted to pass.' Below this is a 'Confidence' bar at 99.0%. At the bottom of the card, it says 'Using: LogisticRegression (test accuracy 85.0%)'.

# Insights and Training, Built In

No terminal required — metrics, charts, and retraining all live inside the app.



Model Insights — accuracy, precision, recall, F1, ROC-AUC + charts



# Built With



**Python 3**

Core language



**scikit-learn**

Models & evaluation



**pandas / NumPy**

Data handling



**Matplotlib**

Charts & reports



**Tkinter**

Desktop GUI



**ttkbootstrap**

Modern UI theming



**joblib**

Model persistence



**Pillow**

Image rendering

# What's in the Project

```
student-performance-ai/  
├─ data/dataset.csv  
├─ models/  
│   ├─ best_model.joblib  
│   ├─ scaler.joblib  
│   └─ model_meta.json  
├─ reports/  
│   ├─ confusion_matrix.png  
│   └─ model_comparison.png  
├─ src/  
│   ├─ generate_dataset.py  
│   ├─ train_model.py  
│   ├─ predict.py  
│   └─ gui_app.py  
├─ requirements.txt  
└─ README.md
```

## How to Run

1

```
pip install -r requirements.txt
```

2

```
python src/train_model.py
```

3

```
python src/gui_app.py
```

*All scripts resolve paths relative to their own location — run them from anywhere.*

Requirements: numpy, pandas, scikit-learn, matplotlib, joblib, ttkbootstrap, pillow

# What's Next

The pipeline is solid — these are the natural next steps to make it production-ready.



## Real Class Data

Swap the synthetic generator for actual semester records



## Per-Student Explanations

Add SHAP values so a prediction shows its “why”



## Standalone App

Package with PyInstaller — no Python install needed



## Track Model Drift

Re-evaluate each semester as real outcomes come in

**Thank you**